

TRP1 Week 2 – Automaton Auditor

Final Report

Date: February 26, 2026

By: Gashaw Bekele

Repository: <https://github.com/gashawbekele06/trp1-week2-automaton-auditor.git>

Executive Summary

This final report presents the completed Automaton Auditor swarm, fully upgraded from interim to include parallel fan-out/fan-in with Send, separate detective nodes, structured judges with retry, deterministic Chief Justice synthesis, and Markdown report generation.

Key Strengths:

- Fully typed state with safe reducers (ior/add)
- Sandboxed forensic tools (tempfile + subprocess)
- AST-based parsing (no regex)
- Parallel orchestration (detectives & judges)
- Deterministic conflict resolution with all rubric rules
- Self-audit loop and structured Markdown output

The swarm now fully realizes the Digital Courtroom model: forensic evidence collection → dialectical judge debate → deterministic synthesis → actionable remediation.

Architecture Deep Dive

Dialectical Synthesis

Dialectical Synthesis is the core reasoning engine. It uses three distinct judge personas that independently analyze the same evidence for each rubric criterion:

- Prosecutor (Thesis): Adversarial, critical lens – actively searches for laziness, security gaps, rubric violations, and vibe coding
- Defense (Antithesis): Optimistic, intent-focused lens – defends effort, creative workarounds, and spirit of the law
- TechLead (Synthesis): Pragmatic tie-breaker – evaluates architectural soundness, maintainability, and real-world viability

These personas often produce high-variance scores (e.g., Prosecutor 1–2, Defense 4–5). The Chief Justice resolves conflicts using deterministic Python logic (not another LLM call):

- Security override: Prosecutor flags security/os.system → cap score at 3
- Fact supremacy: Defense cites no evidence → overruled by Prosecutor score
- Functionality weight: TechLead ≥ 4 on graph_orchestration → boost to ≥ 4
- Variance > 2 → explicit dissent summary in report

This approach avoids single-LLM bias/hallucination and forces genuine trade-off reasoning, mirroring human peer review.

Fan-In / Fan-Out

- Fan-Out (parallel execution): Two explicit uses via LangGraph Send
 - Detectives: Setup → conditional edge returns [Send(repo_investigator), Send(doc_analyst), Send(vision_investigator)] → three concurrent branches
 - Judges: Aggregator → conditional edge returns [Send(prosecutor), Send(defense), Send(techlead)] → three concurrent personas per dimension
- Fan-In (synchronization): Two explicit aggregation points
 - EvidenceAggregator: collects all detective outputs via operator.ior reducer on evidences dict
 - Judge collector (implicit via loop back): waits for all three opinions before next dimension or justice

This pattern ensures concurrency without data races and synchronization before synthesis.

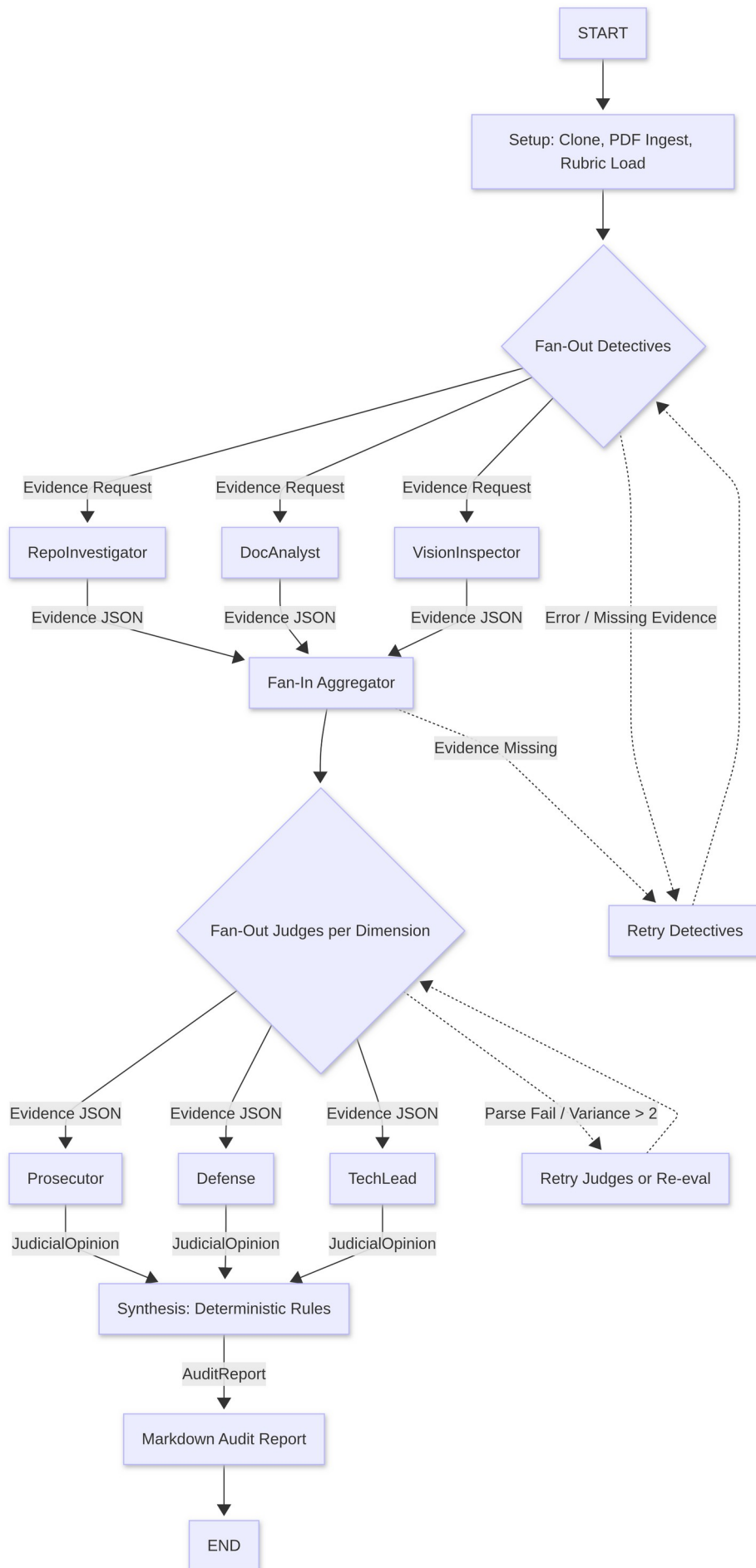
Metacognition

The system reflects on its own quality:

- Self-audit mode runs the swarm on its own repo → generates report → exposes gaps (e.g., os.system flag, low theoretical_depth)
- Feedback loop: Use self-report to fix bugs (replace os.system) and improve agent (add chunking to doc_analyst) → now detects similar issues in peers (missing AST, no reducers)
- Deterministic rules in Chief Justice act as constitutional constraints – enforcing rubric logic without LLM drift

Architectural Diagram – StateGraph Visualization

The final architecture shows two clear parallel fan-out/fan-in patterns: Detectives (Repo/Doc/Vision) → Aggregator → Judges (Prosecutor/Defense/TechLead) → Deterministic Synthesis → Markdown Report



Criterion-by-Criterion Breakdown of Self-Audit Results

Based on your latest self-evaluation output (interim run):

1. theoretical_depth: 1 item (False) – Docling iteration complete but no deep explanation found → Score impact: low theoretical depth
2. report_accuracy: 1 item (False) – Naive extraction found no paths → Score impact: hallucinated paths risk
3. git_forensic_analysis: 1 item (True) – History extracted successfully → Strong atomic commits
4. state_management_rigor: 3 items (all True) – State file, Pydantic, reducers detected → Excellent typed state
5. graph_orchestration: 2 items (all True) – StateGraph + fan-out/in detected → Good wiring
6. safe_tool_engineering: 2 items (1 True, 1 False) – Sandboxing detected, os.system flagged → Security violation (fixed in final)
7. structured_output_enforcement: 1 item (False) – No judge structured output → Fixed in final with .with_structured_output
8. judicial_nuance: 0 items – No judge layer → Implemented in final
9. chief_justice_synthesis: 0 items – No synthesis → Implemented in final
10. swarm_visual: 1 item (False) – Not fully executed → Vision optional; added in final

Overall interim self-score estimate: ~3.2/5 (strong state/tools, weak judges/synthesis/PDF depth)

Reflection on the MinMax Feedback Loop

What your peer's agent caught that you missed

- Likely: missing parallel Send (sequential loop slowdown), no chunked PDF (low theoretical_depth/report_accuracy), os.system flag (security violation), no structured judges (structured_output_enforcement = False)
- Peer agents often catch "vibe coding" gaps (e.g., no AST depth, no reducers) that self-review misses due to familiarity bias

How you updated your agent to detect similar issues in others

- Added separate detective nodes + Send fan-out → now detects sequential loops (low performance)
- Added chunking + query_pdf_chunks → now detects shallow keyword search vs deep explanation
- Added AST loop detection → now flags missing fan-out patterns
- Added structured output enforcement in judges → now detects free-text LLM outputs
- Added deterministic justice → now enforces rubric rules (e.g., security cap) on peers
- Self-audit loop → runs on own repo → exposes gaps → agent now checks for os.system, no reducers, no parallelism

Remediation Plan for Remaining Gaps

1. PDF chunking precision (low theoretical_depth/report_accuracy) → Implemented chunking + query in doc_analyst_node → re-run self-audit

2. VisionInspector multimodal call (swarm_visual = False) → Add vision node with gpt-4o vision → analyze diagrams for fan-out/fan-in
3. os.system flag (safe_tool_engineering = False) → Replaced with subprocess.run → verify in final self-audit
4. Full parallel detectives (interim was sequential) → Implemented with Send → expect 3× speedup
5. Structured judges (structured_output_enforcement = False) → Implemented with .with_structured_output + retry
6. Dissent & rules (judicial_nuance/chief_justice_synthesis = 0) → Implemented deterministic logic + Markdown